Máximos, mínimos y ceros de una función

Capítulo 3 · Arreglos y memoria dinámica

Carlos Pinilla

pinilla.carlos@correounivalle.edu.co

Contenido

1. Radiación de cuerpo negro: leyes de Planck, Stefan-Boltzmann y Wien.
2. Ceros de una función: bisección, Newton-Raphson y secante.
3. Orden de convergencia y criterios de parada.
4. Máximos: analítico, cero numérico y spline de la derivada.
5. Taller 8 y 9: la radiación de cuerpo negro.

Continuidad con la Semana 7

Semana 7

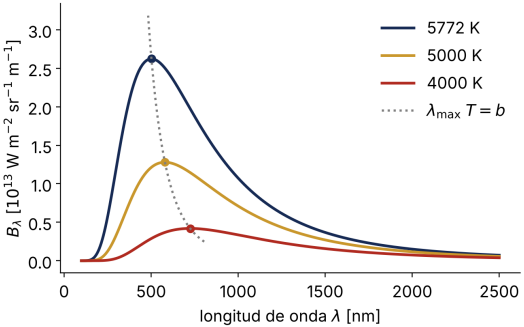
- Integrales: Romberg con parada por la diagonal.
- Derivadas: diferencias finitas sobre arreglos.
- Spline: interpolación entre los puntos de una malla.

Semana 8

- Métodos **iterativos**: una sucesión $x_0, x_1, \ldots$ que se acerca a la solución.
- Un máximo es un cero de la derivada.
- La integral de Planck usa Romberg de nuevo.

Ceros en Fortran; máximos con spline en Python.

Ley de Planck



$$B_\lambda(\lambda, T) = \frac{2hc^2}{\lambda^5} \frac{1}{e^{hc/(\lambda k_B T)} - 1}$$

- B_λ: potencia por unidad de área, de ángulo sólido y de longitud de onda.
- λ: longitud de onda; T: temperatura.
- h: constante de Planck; c: rapidez de la luz; k_B: constante de Boltzmann.

Con $x = hc/(\lambda k_B T)$, energía del fotón sobre energía térmica:

$$B_\lambda \propto \frac{x^5}{e^x - 1}$$

Ley de Stefan-Boltzmann: una integral impropia

Potencia total emitida por unidad de área:

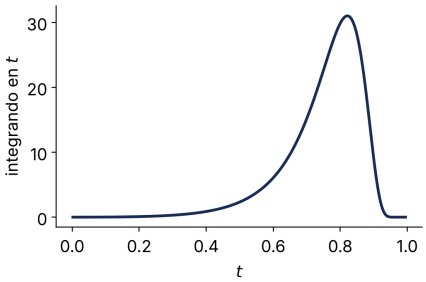
$$M = \int_0^\infty \pi B_\lambda \, d\lambda = \sigma T^4$$

$$\sigma = \frac{2\pi k_B^4}{h^3 c^2} I, \qquad I = \int_0^\infty \frac{x^3}{e^x - 1}\, dx = \frac{\pi^4}{15}$$

σ: constante de Stefan-Boltzmann; I: integral pura, sin unidades.

Romberg necesita un intervalo finito. Con $x = t/(1-t)$:

$$I = \int_0^1 \frac{x(t)^3}{e^{x(t)} - 1} \frac{dt}{(1-t)^2}$$



El integrando tiende a cero en $t = 0$ y en $t = 1$: esos límites se programan aparte.

Ley de Wien: el máximo lleva a una ecuación sin fórmula

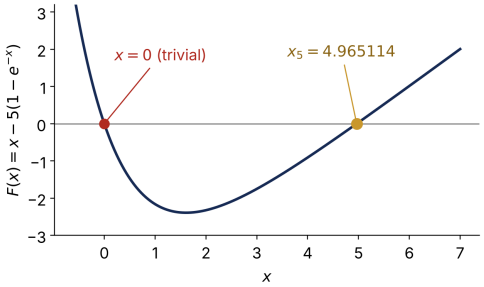
Derivando $x^5/(e^x-1)$ e igualando a cero:

$$\frac{x^4\,[5(e^x-1)-xe^x]}{(e^x-1)^2}=0$$

$$\Longrightarrow \quad x=5\left(1-e^{-x}\right)$$

Ecuación **trascendente**: no tiene solución en funciones elementales. Su raíz positiva fija la constante de Wien:

$$\lambda_{\text{máx}}\,T=b=\frac{hc}{k_B\,x_5}$$



Hallar el máximo se convirtió en hallar un cero:

$$F(x)=0.$$

Ceros de una función

Un **cero** o **raíz** de f es un valor α tal que

$$f(\alpha) = 0.$$

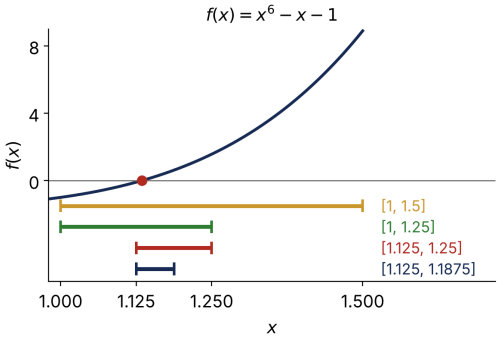
- Hay fórmula para pocos casos: $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$.
- Algunos polinomios se factorizan: $x^3 - 6x^2 + 11x - 6 = (x-1)(x-2)(x-3)$.
- Lo general es aproximar con una sucesión $x_0, x_1, x_2, \ldots$

Teorema del valor intermedio

Si f es continua en $[a, b]$ y $f(a)\, f(b) < 0$, entonces f tiene al menos un cero entre a y b.

Localización en una malla: con arreglos, se buscan los pares de puntos vecinos con signos distintos: $y_i\, y_{i+1} < 0$.

Método de bisección



Algoritmo, con tolerancia ε:

1. Verificar que $f(a)\,f(b) < 0$.
2. Calcular $c = (a + b)/2$.
3. Si $b - c \leq \varepsilon$: aceptar c y terminar.
4. Si $f(a)\,f(c) \leq 0$: $b = c$; si no, $a = c$. Volver al paso 2.

Solo usa el **signo** de f: se conserva la mitad del intervalo donde f cambia de signo.

Bisección: programa y ejemplo

```
if (f(a)*f(b) >= 0.0_dp) stop 'sin arranque'
do n = 1, max_iter
   c = 0.5_dp*(a + b)          ! punto medio
   if (b - c <= tol) exit      ! parada
   if (f(a)*f(c) <= 0.0_dp) then
      b = c                    ! raíz en [a, c]
   else
      a = c                    ! raíz en [c, b]
   end if
end do
```

$x^6 - x - 1 = 0$ en $[1, 2]$, $\varepsilon = 10^{-3}$.
Raíz: $\alpha = 1.134724138$.

Tras 10 iteraciones, $c = 1.1338$ y el error real es 9.4×10^{-4}.

n	a	b	c	$f(c)$
1	1.0000	2.0000	1.5000	8.8906
2	1.0000	1.5000	1.2500	1.5647
3	1.0000	1.2500	1.1250	−0.0977
4	1.1250	1.2500	1.1875	0.6167
5	1.1250	1.1875	1.1562	0.2333
6	1.1250	1.1562	1.1406	0.0616
7	1.1250	1.1406	1.1328	−0.0196
8	1.1328	1.1406	1.1367	0.0206
9	1.1328	1.1367	1.1348	0.0004
10	1.1328	1.1348	1.1338	−0.0096

Bisección: cota del error y propiedades

Cada iteración divide el intervalo a la mitad:

$$|\alpha - c_n| \leq \frac{b-a}{2^n}$$

El número de iteraciones se conoce **antes** de ejecutar:

$$n \geq \frac{\log\left[(b-a)/\varepsilon\right]}{\log 2}$$

Con $[1,2]$ y $\varepsilon = 10^{-3}$: $n \geq 9.97$, es decir, 10 iteraciones.

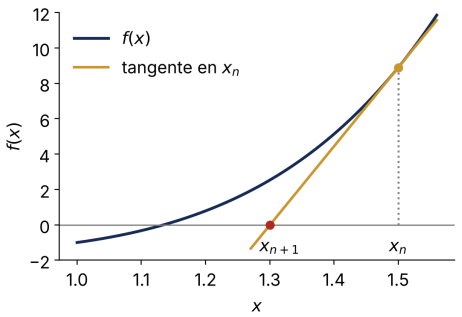
Ventajas

- Converge siempre que el arranque sea válido.
- El error garantizado es conocido.

Limitaciones

- Lenta: una cifra decimal cada 3.3 iteraciones.
- No detecta raíces dobles: $(x-1)^2$ no cambia de signo.

Método de Newton-Raphson



f se reemplaza por su **tangente** en la estimación x_n:

$$y = f(x_n) + f'(x_n)\,(x - x_n)$$

El cruce con $y = 0$ es la siguiente estimación:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

- $f(x_n)$: valor de la función en la estimación actual.
- $f'(x_n)$: pendiente de la tangente.
- Requiere conocer f' y evalúa dos funciones por iteración.

Newton: convergencia cuadrática

Taylor de f alrededor de x_n, evaluada en la raíz:

$$0 = f(x_n) + (\alpha - x_n)f'(x_n) + \tfrac{1}{2}(\alpha - x_n)^2 f''(c_n)$$

con c_n entre α y x_n. Dividiendo por $f'(x_n)$:

$$\alpha - x_{n+1} = (\alpha - x_n)^2 \left[\frac{-f''(c_n)}{2f'(x_n)}\right]$$

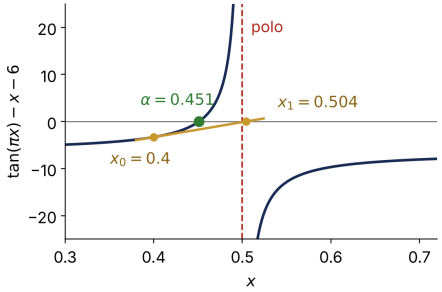
El error siguiente es proporcional al **cuadrado** del error actual. Además, $\alpha - x_n \approx x_{n+1} - x_n$: la diferencia entre iteraciones mide el error.

$$x^6 - x - 1 = 0, \quad x_0 = 1.5$$

n	x_n	$f(x_n)$	$\alpha - x_{n-1}$
0	1.5	8.89	
1	1.30049	2.54	-3.65×10^{-1}
2	1.18148	5.38×10^{-1}	-1.66×10^{-1}
3	1.13946	4.92×10^{-2}	-4.68×10^{-2}
4	1.13478	5.50×10^{-4}	-4.73×10^{-3}
5	1.13472	7.11×10^{-8}	-5.35×10^{-5}
6	1.13472	1.11×10^{-15}	-6.92×10^{-9}

Cerca de la raíz, el exponente del error se duplica en cada iteración.

Newton: dependencia del valor inicial



Desde $x_0 = 0.4$, la tangente cruza el eje al otro lado del polo.

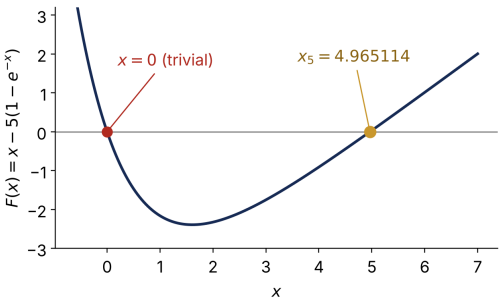
$$f(x) = \tan(\pi x) - x - 6, \quad \alpha = 0.4510473$$

n	$x_0 = 0.48$	$x_0 = 0.40$	$x_0 = 0.01$
1	0.4681700	0.5041505	2.7976098
2	0.4570355	0.5086528	5.2724798
3	0.4517795	0.5188342	6.8756531
4	0.4510582	0.5449097	11.844096
5	0.4510473	0.6309631	17.900251
6	0.4510473	1.1095881	27.701778

Sin garantía de convergencia

Newton puede diverger o llegar a otra raíz. Se fija un máximo de iteraciones y se revisa $|f(x)|$ al terminar.

Valor inicial en la ecuación de Wien



$F(x) = x - 5(1 - e^{-x})$ tiene dos raíces.

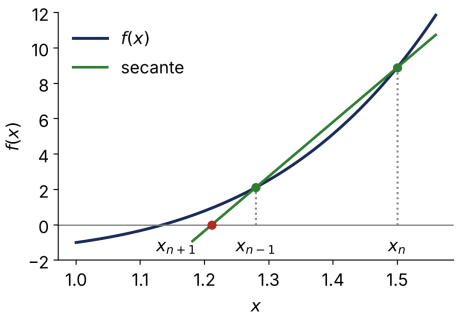
Desde $x_0 = 0.5$, la pendiente es $F'(0.5) = -2.03$: la tangente envía la iteración a $x_1 = -0.22$, y Newton converge a $x = 0$.

Esa raíz es correcta para la ecuación, pero corresponde a $\lambda \to \infty$, donde $B_\lambda \to 0$.

Estrategia combinada

Unas iteraciones de bisección mejoran el valor inicial. Para conservar la garantía, un método híbrido vuelve a bisección cuando el paso de Newton no es adecuado.

Método de la secante

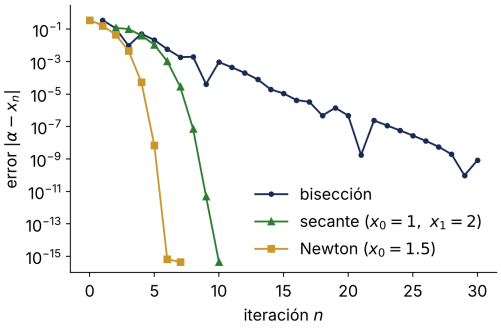


La tangente se reemplaza por la recta que pasa por las dos últimas estimaciones:

$$x_{n+1} = x_n - f(x_n)\,\frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

- El cociente es el inverso de la pendiente de la secante.
- No requiere f'.
- Necesita dos valores iniciales.
- Una sola evaluación nueva de f por iteración.
- Orden $p = (1+\sqrt{5})/2 \approx 1.618$ en una raíz simple.

Orden de convergencia



Un método tiene orden p si, cerca de la raíz,

$$e_{n+1} \approx C\, e_n^p, \qquad e_n = |\alpha - x_n|$$

C: constante que depende de f.

Con tres errores consecutivos:

$$p \approx \frac{\ln(e_{n+1}/e_n)}{\ln(e_n/e_{n-1})}$$

Método	p teórico	p medido
Bisección	1 (cota)	–
Secante	1.618	1.60
Newton	2	2.00

Criterios de parada

Criterio	Condición	Qué controla
Intervalo	$b - c \leq \varepsilon$	Error garantizado en bisección
Paso	$\lvert x_{n+1} - x_n \rvert < \varepsilon$	Error estimado en Newton y secante
Residuo	$\lvert f(x_n) \rvert < \varepsilon$	Cercanía de f a cero
Iteraciones	$n \geq n_{\text{máx}}$	Evita un ciclo sin fin

Un residuo pequeño no basta

Si f es casi plana cerca de la raíz, $f(x)$ puede ser diminuto lejos de α.

Regla del curso

Parar por intervalo o por paso, limitar las iteraciones y revisar el residuo al final.

Comparación de los métodos

	Bisección	Newton	Secante
Datos iniciales	$[a, b]$ con cambio de signo	x_0	x_0, x_1
Requiere	signo de f	f y f'	f
Evaluaciones por iteración	1	2	1
Orden	1	2	1.618
Convergencia garantizada	sí	no	no

Los programas de bisección y Newton están en el notebook, con los ejemplos de estas láminas; la secante es el primer ejercicio de calentamiento.

Máximos y mínimos: analítico

Candidatos a extremo interior de una función suave:

$$f'(x^*) = 0$$

$f''(x^*) < 0$: máximo, $\quad f''(x^*) > 0$: mínimo

$f' = 0$ no distingue entre los dos. Las condiciones sobre f'' son suficientes, no necesarias: x^4 tiene un mínimo en cero con $f'' = 0$.

Ejemplo: función proporcional a la distribución de Maxwell-Boltzmann, en unidades de $\sqrt{k_B T/m}$:

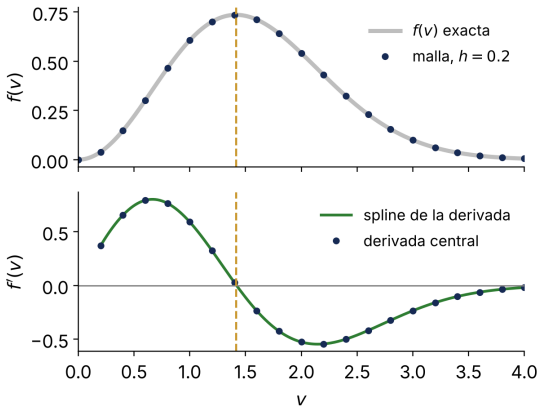
$$f(v) = v^2\, e^{-v^2/2}$$

$$f'(v) = \left(2v - v^3\right) e^{-v^2/2} = 0$$

$$\implies v^* = \sqrt{2} = 1.414214$$

v^* es la rapidez más probable. Sirve de referencia para los métodos numéricos.

Máximo numérico: cero de la derivada en una malla



Datos en una malla de paso h:

1. **Mayor valor del arreglo**: error de orden h.
2. **Cero lineal**: derivada central d_i; se interpola entre el par con $d_i > 0$ y $d_{i+1} \leq 0$.
3. **Spline de la derivada**: cero del spline que pasa por los d_i.

La derivada del spline en ese cero aproxima f'' e indica si es un máximo.

Spline cúbico en Python

```
from scipy.interpolate import CubicSpline

d = (y[2:] - y[:-2])/(2*h)      # derivada central
s_d = CubicSpline(v[1:-1], d)   # spline de d

for r in s_d.roots(extrapolate=False):
    print(r, s_d(r, 1))         # cero y f''
```

Con $h = 0.2$: $v^* = 1.418953$, $f'' \approx -1.42$: máximo.

- Un polinomio de grado 3 en cada intervalo.
- Valor, primera y segunda derivada continuas en las uniones.
- Pasa exactamente por los datos.

Por qué SciPy

Construir el spline requiere un sistema tridiagonal, tema del Capítulo 5.

Precisión al refinar la malla

Error en $v^* = \sqrt{2}$

h	Arreglo	Spline de d	Spline de f
0.40	1.9×10^{-1}	2.0×10^{-2}	2.5×10^{-4}
0.20	1.4×10^{-2}	4.7×10^{-3}	8.8×10^{-5}
0.10	1.4×10^{-2}	1.2×10^{-3}	2.9×10^{-5}
0.05	1.4×10^{-2}	2.9×10^{-4}	4.1×10^{-6}

Spline de d: spline de la derivada central. Spline de f: derivada del spline de los datos.

- El arreglo no mejora: $v = 1.4$ sigue siendo el punto de mayor valor.
- El spline de d baja con razón 4: orden 2.

El eslabón más débil

El spline es muy preciso, pero la derivada central que lo alimenta es de orden 2 y fija el error.

Taller 8 y 9: la radiación de cuerpo negro

Parte	Qué se hace
B	Fortran: módulo con bisección, Newton y secante; constante de Wien
C	Fortran: Romberg del Taller 7 con un integrando nuevo; constante de Stefan-Boltzmann
D	Python: máximo del espectro solar en una malla, con spline

Entrega: viernes 20 de noviembre.

Lunes 9 de noviembre

- Explicación del enunciado.
- Ejercicios de calentamiento del notebook: secante, barrido de un polinomio y spline en una malla gruesa.

El enunciado completo y la celda de verificación con cinco casos están en el notebook.

Conclusiones

- Una ecuación sin fórmula se resuelve con una **sucesión**: cada método es una regla para pasar de una estimación a la siguiente.
- Bisección garantiza la convergencia; Newton y la secante ganan velocidad a cambio de esa garantía.
- El **orden** mide la rapidez: Newton duplica las cifras correctas en cada iteración cerca de la raíz.
- Un programa de ceros necesita criterio de parada, máximo de iteraciones y revisión del resultado.
- Un máximo se busca como cero de f' con $f'' < 0$; la precisión del resultado la fija el eslabón más débil del cálculo.

Próxima semana: registros, clases y bibliotecas.

Referencias

- M. T. Heath, *Scientific Computing: An Introductory Survey*, 2.ª ed. revisada, SIAM, 2018, caps. 5 y 6.
- R. H. Landau, M. J. Páez y C. C. Bordeianu, *Computational Physics: Problem Solving with Python*, 4.ª ed., Wiley.
- S. J. Chapman, *Fortran for Scientists and Engineers*, 4.ª ed., McGraw-Hill, 2018.
- C. Pinilla, *Finding roots of a function*, notas de curso.